LAB 11 – AdaBoost Implementation

Import Libraries

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, roc_auc_score
from sklearn.preprocessing import LabelEncoder
```

Load Dataset

```
df = pd.read_csv("/content/data.csv")

df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean
0	842302	M	17.99	10.38	122.80	1001.0
1	842517	M	20.57	17.77	132.90	1326.0
2	84300903	M	19.69	21.25	130.00	1203.0
3	84348301	M	11.42	20.38	77.58	386.1
4	84358402	M	20.29	14.34	135.10	1297.0

5 rows × 33 columns

Drop Unnecessary Columns

```
df = df.drop(['id', 'Unnamed: 32'], axis=1)
```

Encode Target (Malignant / Benign)

```
le = LabelEncoder()

df['diagnosis'] = le.fit_transform(df['diagnosis'])
```

```
# M = 1 , B = 0
df.head()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothnes
0	1	17.99	10.38	122.80	1001.0	(
1	1	20.57	17.77	132.90	1326.0	(
2	1	19.69	21.25	130.00	1203.0	(
3	1	11.42	20.38	77.58	386.1	(
4	1	20.29	14.34	135.10	1297.0	(

5 rows × 31 columns

Define Features and Target

```
X = df.drop('diagnosis', axis=1)
y = df['diagnosis']
```

Train/Test Split (80/20)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

Train Decision Tree Classifier

```
dt_model = DecisionTreeClassifier(random_state=42)

dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)

acc_dt = accuracy_score(y_test, y_pred_dt)
roc_dt = roc_auc_score(y_test, y_pred_dt)

print("Decision Tree Results")
print("Accuracy:", round(acc_dt,4))
print("ROC-AUC:", round(roc_dt,4))
```

```
Decision Tree Results
Accuracy: 0.9474
ROC-AUC: 0.944
```

Train AdaBoost Classifier

```
ada_model = AdaBoostClassifier(
    n_estimators=50,
    random_state=42
)

ada_model.fit(X_train, y_train)

y_pred_ada = ada_model.predict(X_test)

acc_ada = accuracy_score(y_test, y_pred_ada)
roc_ada = roc_auc_score(y_test, y_pred_ada)

print("AdaBoost Results")
print("Accuracy:", round(acc_ada,4))
print("ROC-AUC:", round(roc_ada,4))
```

```
AdaBoost Results
Accuracy: 0.9649
ROC-AUC: 0.9581
```

Compare Both Models

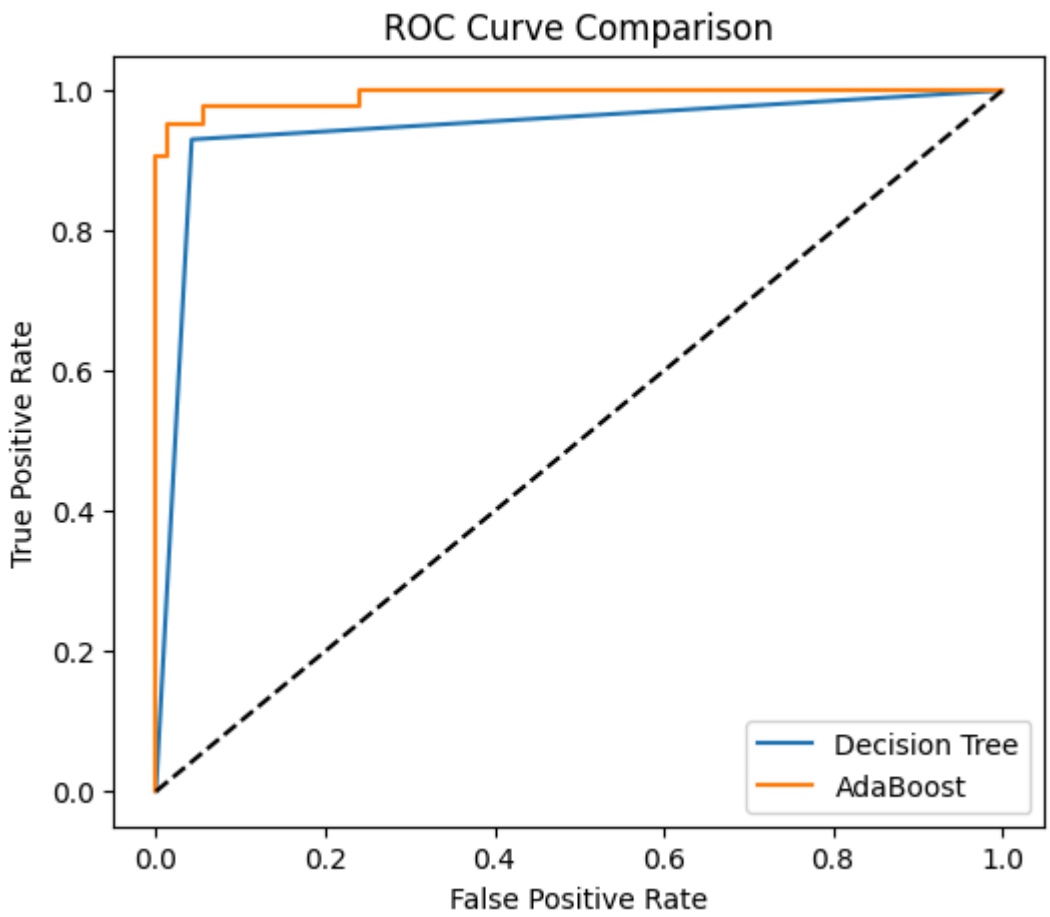
```
print("\nModel Comparison")

print("\nDecision Tree:")
print("Accuracy:", round(acc_dt,4))
print("ROC-AUC:", round(roc_dt,4))

print("\nAdaBoost:")
print("Accuracy:", round(acc_ada,4))
print("ROC-AUC:", round(roc_ada,4))
```

```

Model Comparison

Decision Tree:
Accuracy: 0.9474
ROC-AUC: 0.944

AdaBoost:
Accuracy: 0.9649
ROC-AUC: 0.9581
```

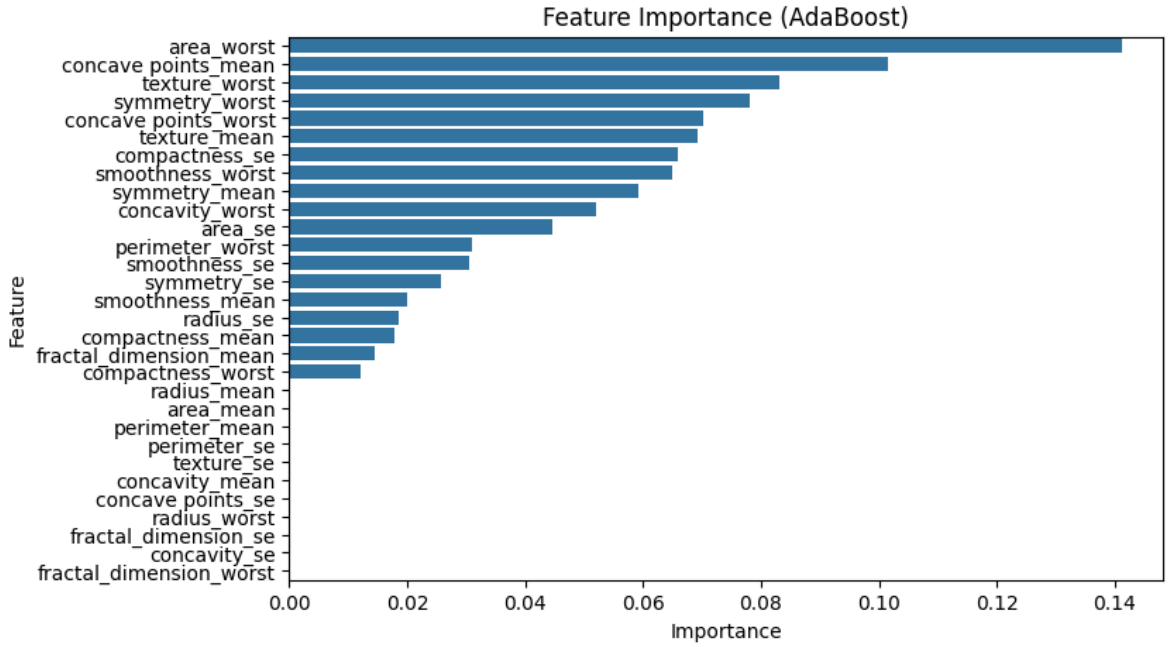
ROC Curve Graph

```
import matplotlib.pyplot as plt
from sklearn.metrics import roc_curve
```

```
# Decision Tree probabilities
y_prob_dt = dt_model.predict_proba(X_test)[:,1]

# AdaBoost probabilities
y_prob_ada = ada_model.predict_proba(X_test)[:,1]

# Compute ROC curve
fpr_dt, tpr_dt, _ = roc_curve(y_test, y_prob_dt)
fpr_ada, tpr_ada, _ = roc_curve(y_test, y_prob_ada)

plt.figure(figsize=(6,5))

plt.plot(fpr_dt, tpr_dt, label="Decision Tree")
plt.plot(fpr_ada, tpr_ada, label="AdaBoost")

plt.plot([0,1],[0,1],'k--')

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()

plt.show()
```



Feature Importance Graph

```
import seaborn as sns
```

```
importance = ada_model.feature_importances_

feature_names = X.columns

importance_df = pd.DataFrame({
    "Feature": feature_names,
    "Importance": importance
})

importance_df = importance_df.sort_values(by="Importance", ascending=False

plt.figure(figsize=(8,5))
sns.barplot(x="Importance", y="Feature", data=importance_df)

plt.title("Feature Importance (AdaBoost)")
plt.show()
```